

A Help Desk Ticket Management System:

Multi-User modern system with a dashboard design built for IT customer support teams that need real-time visibility into ticket analytics, agent performance, and support operations.

Dashboard:

The dashboard should combine **ticket tracking**, user **activity analysis**, category distribution, and user workload insights into one **clear admin panel UI**, helping support managers make faster, data-driven decisions without switching tools.

Designed with a **clean layout**, clear data hierarchy, and **intuitive charts**, that has a customer support dashboard which focuses on clarity, speed, and usability for daily support operations. And should be responsive design

The System:

The system should have configurable panel or pages for admins and managers to setup KPI for teams and users for which they can also track all KPI and daily jobs assigned to team members or users within the team or generally.

1. System Overview

1.1 Purpose

A multi-tenant help desk and performance management system designed for IT support teams, enabling:

- Real-time ticket tracking
- Team-based queue management with controlled ownership
- Engineer workload balancing
- KPI-driven performance tracking
- Advanced reporting and analytics

1.2 Core Design Principles

- ⚡ Fast decision-making (real-time + pre-aggregated data)
 - 📦 Reliability and auditability
 - 📊 KPI-first architecture
 - 🏢 Multi-tenant ready (internal + SaaS)
 - ⚖️ Balanced workload via queue intelligence
-

2. Ticket Assignment Model (FINALIZED)

The system uses a **hybrid queue-based assignment model**:

- ✓ Team-based queue
 - ✓ Controlled pickup
 - ✓ Auto-assignment fallback
 - ✓ SLA-driven escalation
-

2.1 Ticket Lifecycle

NEW

- TEAM_ASSIGNED
 - UNASSIGNED
 - ASSIGNED
 - IN_PROGRESS
 - ESCALATED
 - RESOLVED
 - CLOSED
-

2.2 Assignment Flow

Phase 1: Team Routing

- Ticket assigned to a **Team**
 - No engineer assigned yet
-

Phase 2: Controlled Pickup

- Engineers can **claim tickets**
 - System enforces:
 - Max workload
 - Queue visibility
 - Priority ordering
-

Phase 3: Auto-Assignment

If ticket not picked within SLA:

- System assigns automatically:
 - Least loaded engineer

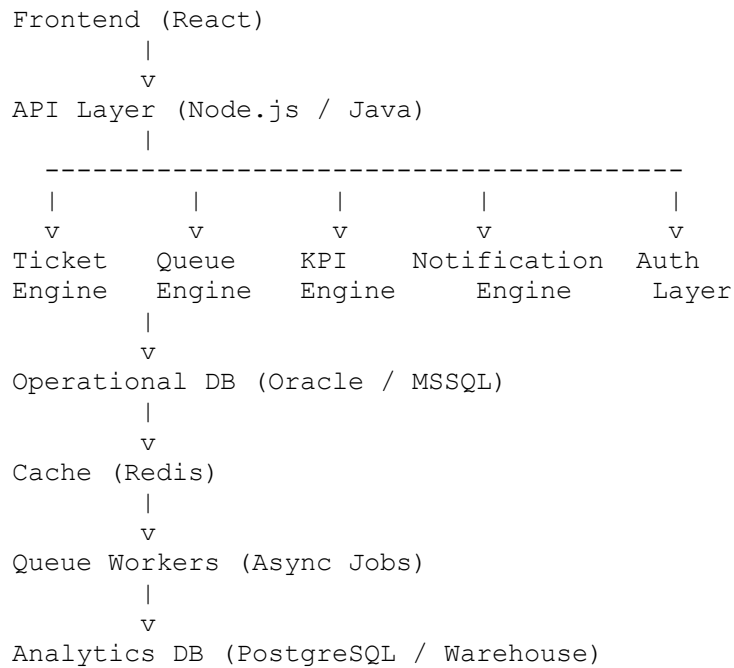
- Round-robin fallback

Phase 4: SLA Escalation

- Pickup SLA breach → auto-assign + notify
- Resolution SLA breach → escalate + alert

3. System Architecture

3.1 High-Level Architecture



3.2 Core Components

◆ Ticket Engine

- Ticket lifecycle management
- Comments, attachments

◆ Queue Engine (Critical Component)

- Team queues
 - Ticket ordering (priority + SLA)
 - Pickup SLA tracking
 - Auto-assignment logic
-

◆ KPI Engine

- KPI definition and tracking
 - Real-time + batch computation
-

◆ Notification Engine

- Email + in-app alerts
 - SLA breach notifications
-

◆ Analytics Layer

- Aggregated data for reporting
 - Fast dashboard queries
-

4. Functional Modules

4.1 Ticket Management

- Create, update, resolve tickets
 - Categorization and prioritization
-

4.2 Queue Management

- Team queues
- Controlled ticket pickup
- Auto-assignment
- SLA enforcement

4.3 Task Management

- Assign non-ticket work
- Daily workload tracking

4.4 KPI Management

- Define KPIs per:
 - Engineer
 - Team
- Track:
 - Productivity
 - SLA compliance

4.5 Dashboard Module

Key Sections:

- Executive Summary
- Queue Health
- Ticket Analytics
- Engineer Performance
- Workload Distribution
- Real-time Activity Feed

4.6 Reporting Module

Report Categories:

◆ Operational Reports

- Open tickets
- SLA breaches
- Ticket aging

◆ Performance Reports

- Engineer productivity
 - KPI achievement
-

◆ Queue Reports

- Pickup time
 - Queue backlog
 - Unassigned trends
-

◆ Management Reports

- Monthly trends
 - Team comparisons
-

Features:

- Filters (date, team, category)
 - Export (PDF, Excel)
 - Scheduled reports
-

5. SLA Management

5.1 Pickup SLA

- Time allowed before ticket must be claimed

Actions on breach:

- Auto-assignment
 - Manager notification
-

5.2 Resolution SLA

- Time allowed to resolve ticket

Actions on breach:

- Escalation
- Alerts

6. Multi-Tenancy Design

6.1 Tenant Model

- Each organization = Tenant
- All tables include:
 - `tenant_id`

6.2 Isolation Levels

- Shared DB (SaaS default)
- Dedicated DB (enterprise option)

6.3 Tenant Configurations

- SLA rules
- KPI definitions
- Workflow rules

7. Data Model (High-Level)

Core Entities:

- Tenants
- Users (Engineers)
- Roles
- Teams
- Tickets

- Ticket_Comments
 - Tasks
 - KPIs
 - KPI_Results
 - Notifications
 - Audit_Logs
 - Assignment_History
 - Queue_Config
-

8. Performance Strategy

8.1 Caching (Redis)

- Dashboard metrics
 - Queue counts
 - KPI summaries
-

8.2 Pre-Aggregation

- Daily KPI results
 - Ticket summaries
-

8.3 Indexing

- ticket_status
 - assigned_engineer
 - created_at
 - priority
-

8.4 Async Processing

- SLA timers
 - Notifications
 - Auto-assignment
-

9. Reliability & Availability

- Retry mechanisms for jobs
 - Persistent SLA timers
 - Database backups
 - Audit logs for all actions
-

10. Security

- Role-Based Access Control (RBAC)
 - Tenant data isolation
 - Encryption (HTTPS, at rest)
-

11. Integration Capabilities

- Email → ticket creation
 - SMS alerts
 - API integrations
 - Future:
 - AI modules
 - Chat systems
-

12. Differentiation

Compared to tools like Zendesk and Freshdesk:

- Queue intelligence (not just visibility)
 - KPI-first architecture
 - Real-time operational control
 - Hybrid deployment (internal + SaaS)
-

13. Future Enhancements

- AI ticket classification
- SLA breach prediction
- Smart workload balancing

- Chatbot integration

1. MSSQL – OPERATIONAL SCHEMA

This handles:

- Tickets
 - Queue logic
 - SLA
 - Users
 - KPI definitions
 - Audit logs
-

◆ 1.1 Core Design Principles

- Multi-tenant → `tenant_id` everywhere
- High-write optimized
- Indexed for:
 - queue queries
 - SLA tracking
 - dashboard reads

2. POSTGRESQL – ANALYTICS SCHEMA

2.1 DESIGN PRINCIPLES

- No transactional updates
- Pre-aggregated data
- Optimized for dashboards

PERFORMANCE OPTIMIZATION

MSSQL

- Partition `tickets` by date (monthly)
 - Archive old tickets
 - Use covering indexes
-

PostgreSQL

- Use materialized views for dashboards
- Precompute metrics
- Index by:
 - date
 - tenant_id